찾아라! 도깨비 — MVP 아키텍처

개발할 때 옆에 두고 보는 문서. 어떤 파일이 무엇을 하고, 무엇을 바꾸려면 어디를 고치는지.

1. 한눈에

엔진	Unity 6.3 LTS (6000.3.11f1), uGUI + TextMeshPro, Input System. 씬 1개 `Assets/Scenes/Game.unity`
씬 만들기	씬은 손으로 만들지 않고 `MvpSceneBuilder`가 코드로 전부 만들어요 (메뉴 Tools → 찾아라도깨비 → MVP 씬 만들기). 배치를 바꾸려면 이 파일을 고치고 다시 만들기
화면	패널 8개 (시작 · 도입 · 관찰 · 그리기 · 판정 · 결과 · 비교 · 종료) 중 하나만 켜고, 위에 HUD · 일시정지. 화면 기준 1080×1899 = 팀 시안(2248×3952)을 줄인 크기, 시안 픽셀 좌표 그대로 배치
판정	멀티모달 LLM = Google Gemini (`gemini-3.5-flash`, 무료 등급 : 하루 약 20번). 웹 · 에디터 모두 판정 중계 서버(Cloudflare Worker)를 거쳐요 (키는 서버에만). 프롬프트는 기획서 부록 B 문장 그대로. 실패하면 간식 돌려주고 다시 제출
배포	웹 게임 = GitHub Pages (`gh-pages` 브랜치), 판정 서버 = `Server/judge` (wrangler), 코드 = `feature/mvp-stage1`

2. 폴더

```
FindDokkaebi/
  Assets/Scripts/     게임 코드 (아래 3장)
  Assets/Editor/      에디터 도구 : 씬 만들기 · 자동 테스트 · 판정 검사 · 웹 빌드 · 글꼴 굽기
  Assets/Data/        단계 데이터 Stage1~5 (.asset, 씬 만들 때 MvpSceneBuilder 가 채움)
  Assets/Sprites/     TeamUI(시안에서 오린 UI, 절반 크기) · Background · Characters · Stage1~5(원본 · 엉뚱한 물건) · UI(그림 준비 중 등)
  Assets/Fonts/       Jua(시안 글꼴, 구운 SDF) · Noto Sans KR(없는 기호 대신)
  ArtSource/          아트 원본 정리 : 원본(기획서 그림 · 첨부 사진) + 화면별로 들어간 부품 + README 표
  Server/judge/       판정 중계 서버 (worker.js · stages.json 게임 프롬프트 · appendix_b.json 부록 B 원문 · wrangler.toml)
  Docs/               이 문서들 · src(문서 · 그림 만드는 스크립트) · 참고(예전 문서)
  Builds/WebGL/       웹 빌드 결과 (git 에 안 올림)
  UserSettings/OpenAI.json   (선택) 판정 서버 주소가 비어 있을 때만 쓰는 OpenAI 키 — git 에 안 올림
```

3. 게임 코드 (Assets/Scripts)

파일	하는 일	주로 고칠 때
`GameManager`	화면 흐름(상태 머신) · 간식 · 타이머 · 판정 요청 · 결과 규칙 · 기록 · 일시정지	흐름 · 규칙 · 문구
`StageData`	단계 하나의 데이터 (이름 · 이야기 · 규칙 · 색 모드 · 원본 그림 · 특징 목록). `Feature` = 특징 하나 (id · 이름 · 판정 기준 글 · 엉뚱한 물건 그림 · 테스트용 계산 종류)	단계 · 특징 추가
`DrawingBoard`	512×512 쪽지 : 펜 · 지우개 · 굵기 · 되돌리기(20번) · 모두 지우기, 제출할 그림 텍스처. 잉크는 쪽지 안에만 칠해지고, 그리다가 밖으로 나갔다 들어오면 쪽지 안에 걸친 부분만 그려요 (ClipToBoard)	그리기 느낌
`ToolBar`	색 · 연필 · 지우개 · 굵기 버튼, 고른 표시, 단계마다 색 버튼 보이기/숨기기	도구 버튼
`ColorMixer`	색 섞기 카드 : 재료 고르기 → 결과 색 (MixTable) → 섞은 색 칸	섞는 규칙
`OpenAIJudge`	AI 판정 요청 : 판정 서버 주소가 있으면 서버, 없으면 OpenAI.json 키로 OpenAI 직접. 프롬프트(BuildPrompt) = 부록 B 문장 그대로 (검은색 단계 = 방망이 프롬프트, 색 단계 = 돗자리 프롬프트, 단계 이름 · 목표 대상 · 특징만 바꿔 넣음) · 응답 JSON 해석(특징별 present)	프롬프트 · 서버 주소
`LocalJudge`	계산식 판정 (모양 · 색 계산). **자동 테스트 전용** — 실제 플레이에서는 안 씀	테스트
`SimpleAnim`	도깨비 갸웃 · 물건 뽕 · 방망이 휘두르기	연출

화면 흐름 (GameManager.State)

```
Title —StartGame→ StageIntro —StartObserve→ Observe(3초) —자동→ Draw(60초)
Draw —Submit(간식-1)→ Judge —AI 결과→ Result —ShowCompare→ Compare —NextStage→ StageIntro(다음) … → Ending —BackToTitle→ Title
Judge —AI 실패→ (간식+1) —BackToDraw→ Draw(그리던 그림)     Result —Retry(간식 남음)→ Draw(빈 쪽지, 60초)
어느 화면이든 —TogglePause→ 일시정지 (Time.timeScale 0)
```

결과 규칙 (ShowResult)

특징 전부 그림 → 성공 · 진짜 물건 / 일부 → 처음 빠뜨린 특징의 엉뚱한 물건 / 하나도 없음 → 아무것도 못 찾음. 실패했고 간식이 남으면「다시 그리기」.

4. 판정이 흐르는 길

```
제출 → board.Texture → PNG → base64
  ├ (판정 서버 주소 있음 = 보통)  판정 서버 POST /judge {stage, image}
  |      서버 : 허락한 곳(웹 게임 주소 · Unity)만, IP 당 1분 20번 → stages.json 의 그 단계 프롬프트 + 그림
  |            → Google Gemini (GEMINI_MODEL). 서버는 미국(aws:us-east-1)에서 돌아요 (홍콩에서 보내면 Gemini 가 거절)
  └ (서버 주소 없음)  UserSettings/OpenAI.json 키로 OpenAI Responses API 에 바로
← AI JSON {inputStatus, features:[{featureId, status, evidence}]} → present = (status == present) → ShowResult
실패(연결 · 키 · JSON) → OnJudgeError : 간식 돌려주고 「그림으로 돌아가 다시 제출」
```

> 프롬프트를 바꾸면 : 특징 글은 `MvpSceneBuilder.CreateStageDataFiles`, 프롬프트 틀은 `OpenAIJudge.BuildPrompt` → 씬 만들기 → 판정 서버 프롬프트 내보내기(`Server/judge/stages.json`) → `Server/judge` 에서 `npx wrangler deploy`. 프롬프트는 서버에 있어서 웹 게임은 다시 빌드하지 않아도 돼요. 서버 키 : 키를 복사한 뒤 `Get-Clipboard | npx wrangler secret put GEMINI_API_KEY`. 무료 등급은 하루 약 20번 (한국 시간 오후 4시에 다시 채워짐).

판정 검증 (기획서 8장 다시 하기)

부록 A 그림 4장 + 부록 B 프롬프트 원문(`Server/judge/appendix_b.json`)으로 그림마다 한 번씩 판정 → 2026-10-27 Gemini 로 4장 모두 기획서 표와 같은 결과. 입력 그림과 실제 JSON : `Docs/참고/기획서8장_재검증_Gemini_2026-10-27/`. ※ 기획서 8장 돗자리는 십자 네 칸, 게임 3단계 돗자리는 최종 아트대로 색 띠 네 줄이라 프롬프트가 달라요.

> 자동 플레이 테스트(MvpSelfTest)는 AI 를 부르지 않아요. 화면 흐름을 보려고 판정 결과를 미리 정해서 넣고, 그 결과에 맞는 손그림(HandSketch)을 제출해요.

5. 화면 만들기 (MvpSceneBuilder)

좌표	`PlaceM(rt, x0,y0,x1,y1)` = 시안 픽셀 상자 그대로 (K = 1080/2248). `TeamImage` = 오린 그림을 원래 자리에, `TeamTool` = 시안 그림 버튼 + 고른 표시 고리
화면 크기	CanvasScaler 1080×1899 Expand, 가운데 `Frame` 안에 모든 화면. 폰이 더 길면 위아래에 방 배경
그림 설정	시안 UI · 시작 화면은 압축 없이 + 밉맵 + 최대 4096 (외곽선이 또렷하게). 카드 · 버튼은 9-slice (TeamCard · TeamPillCoral · TeamPillCream)
글꼴	Jua SDF (`MvpFontBake` 가 게임 글자만 구움). 글자를 새로 넣었으면 글꼴 굽기를 다시
단계 데이터	`CreateStageDataFiles` : 이야기 · 규칙 · 특징(이름 · 기준 글) · 그림 연결. 씬 만들 때마다 덮어써요

6. 도구 (Tools → 찾아라도깨비, 또는 Temp/ 의 flag 파일)

도구	파일	하는 일
MVP 씬 만들기	MvpSceneBuilder · MvpAutoBuild	씬을 처음부터 만들기 (mvp_rebuild.flag)
자동 플레이 테스트	MvpSelfTest · MvpLayoutCheck	5단계 흐름 · 규칙 · 버튼 · 펜(그림판 밖) 확인 + 화면 캡처(Temp/SelfTest) + 배치 숫자 검사(걸쳐 겹침 · 글자 넘침 · 화면 밖) (mvp_selftest.flag)
판정 전수 검사	MvpJudgeCases · HandSketch	특징 조합 112가지 손그림을 계산식 판정에 넣어 보기 (mvp_judgecases.flag)
글꼴 굽기	MvpFontBake	Jua SDF (mvp_font.flag)
웹 빌드	MvpWebGLBuild	Builds/WebGL + 판정 서버 프롬프트 내보내기 (mvp_webgl.flag)
판정 서버 프롬프트	MvpJudgeServer	stages.json 내보내기, server_url.txt → 씬의 서버 주소
플레이 영상	MvpPlayRecorder	자동 플레이를 프레임으로 저장 (예전 화면 기준, 다시 찍어야 함)

7. 자주 하는 수정

하고 싶은 것	방법
팀 그림이 왔어요 (엉뚱한 물건 · 표정)	`Assets/Sprites/Stage*/Wrong_F*.png` · `Characters/Boy_*.png` 를 같은 이름으로 덮어쓰기 → 씬 만들기 → 웹 빌드
시안 UI 가 바뀌었어요	새 시안을 `ArtSource/원본` 에 → `Docs/src/art_pipeline` 스크립트로 다시 오리기 → TeamUI 덮어쓰기
배치를 옮기고 싶어요	MvpSceneBuilder 의 PlaceM 좌표(시안 픽셀) → 씬 만들기 → 자동 테스트의 배치 검사가 0 인지
특징 · 판정 기준	CreateStageDataFiles 의 특징 글 → 씬 만들기 → 판정 서버 프롬프트 내보내기 → wrangler deploy (결과 화면의 특징 이름도 바뀌었으면 웹 빌드도)
돗자리 온전한 그림이 왔어요	지금 돗자리 그림은 기획서 그림 5 에서 오린 것이라 왼쪽 모서리가 잘려 있어요 (팀에 요청함). 오면 `Assets/Sprites/Stage3_Mat/Original.png` 를 덮어쓰기 → 웹 빌드
웹에 올리기	웹 빌드 → Builds/WebGL/Build 를 gh-pages 브랜치에 커밋 · 푸시 (GitHub Pages)